```
    this.mockWebServer = new MockWebServer();
    this.mockWebServer.start();                                    ❶
    var webClient = WebClient.builder()                            ❷
      .baseUrl(mockWebServer.url("/").uri().toString())
      .build();
    this.bookClient = new BookClient(webClient);
  }

  @AfterEach
  void clean() throws IOException {
    this.mockWebServer.shutdown();                                 ❸
  }
}
```

❶ Starts the mock server before running a test case

❷ Uses the mock server URL as the base URL for WebClient

❸ Shuts the mock server down after completing a test case

Next, in the `BookClientTests` class, you can define some test cases to validate the client's functionality in Order Service.

Listing 8.29 Testing the interaction with the Catalog Service application

```
package com.polarbookshop.orderservice.book;

...
import okhttp3.mockwebserver.MockResponse;
import reactor.core.publisher.Mono;
import reactor.test.StepVerifier;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;

class BookClientTests {
  private MockWebServer mockWebServer;
  private BookClient bookClient;

  ...

  @Test
  void whenBookExistsThenReturnBook() {
    var bookIsbn = "1234567890";

    var mockResponse = new MockResponse()                          ❶
      .addHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
      .setBody("""
        {
          "isbn": %s,
          "title": "Title",
```